

The Maths Behind XGBoost

1) Big Picture — What XGBoost Is Doing

- XGBoost is **gradient boosting** with extra engineering and explicit regularization.
- It builds the model **additively**: each round we add one tree that reduces the current loss.
- Because trees are discrete objects, XGBoost uses a **second-order Taylor approximation** of the loss to get closed-form decisions for leaf weights and split gains.

We will derive: *gradients/hessians* → *per-leaf optimal output* → *split gain (similarity score)* → *update rules for regression/classification*.

2) Notation And Setup

- Dataset: $\{(x_i, y_i)\}_{i=1}^n$.
- Model after $m-1$ rounds: $\hat{y}_i^{(m-1)}$.
- We add a tree f_m , so new prediction:

$$\hat{y}_i^{(m)} = \hat{y}_i^{(m-1)} + \eta f_m(x_i)$$

where η is the learning rate and $f_m(x)$ maps input to a leaf weight.

- Objective (to minimize):

$$\text{Obj}^{(m)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(m)}) + \sum_{t=1}^m \Omega(f_t)$$

where $\Omega(f)$ is the regularization term for tree complexity.

Typical regularizer used in XGBoost (leaf-wise):

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \quad (\text{optionally} + \alpha \sum_j |w_j|)$$

- T = number of leaves in the tree, w_j = weight of leaf j .
 - λ = L2 regularization on leaf weights, γ = penalty per leaf.
-

3) Taylor Expansion Of the Loss (Why and How)

Directly minimizing $\sum_i l(y_i, \hat{y}_i^{(m)})$ w.r.t. tree structure is hard. So XGBoost does a **second-order Taylor expansion** of the loss around current prediction $\hat{y}_i^{(m-1)}$.

For a single example:

$$l(y_i, \hat{y}_i^{(m)}) \approx l(y_i, \hat{y}_i^{(m-1)}) + g_i f_m(x_i) + \frac{1}{2} h_i f_m(x_i)^2$$

where

$$g_i = \left. \frac{\partial l(y_i, \hat{y}_i)}{\partial \hat{y}_i} \right|_{\hat{y}_i = \hat{y}_i^{(m-1)}}, \quad h_i = \left. \frac{\partial^2 l(y_i, \hat{y}_i)}{\partial \hat{y}_i^2} \right|_{\hat{y}_i = \hat{y}_i^{(m-1)}}$$

(These are the **gradient** and **Hessian**—pseudo-responses for the current round.)

Drop constant terms (those independent of f_m) and sum over i . The approximate objective to **optimize w.r.t. the new tree** (and its leaf weights) becomes:

$$\widetilde{\text{Obj}}^{(m)} = \sum_{i=1}^n (g_i f_m(x_i) + \frac{1}{2} h_i f_m(x_i)^2) + \Omega(f_m) + \text{const}$$

This quadratic form is the reason we can get closed-form solutions for leaf weights.

4) Model the Tree as Leaf Weights

Let the tree partition training samples into T leaves. Let $q(x_i) \in \{1, \dots, T\}$ denote the leaf index of x_i . The tree output is constant per leaf:

$$f_m(x_i) = w_{q(x_i)}.$$

Group terms by leaf j . Define for each leaf:

$$G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i$$

where I_j is the set of samples falling into leaf j .

Plugging into the approximate objective (and including $\Omega = \gamma T + \frac{1}{2} \lambda \sum_j w_j^2$):

$$\widetilde{\text{Obj}}^{(m)} = \sum_{j=1}^T \left(G_j w_j + \frac{1}{2} H_j w_j^2 \right) + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 + \text{const}$$

Combine the quadratic terms:

$$\widetilde{\text{Obj}}^{(m)} = \sum_{j=1}^T \left(G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right) + \gamma T + \text{const}$$

↓

This is separable per leaf $\rightarrow$ we can minimize leaf by leaf.

5) Optimal Leaf Weight (Closed Form)

For a fixed leaf j , minimize w.r.t. w_j :

$$\frac{\partial}{\partial w_j} \left(G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right) = 0$$

gives

$$G_j + (H_j + \lambda) w_j = 0 \quad \Rightarrow \quad \boxed{w_j^* = -\frac{G_j}{H_j + \lambda}}$$

This is the **optimal output value** of leaf j (before applying the learning rate η). In practice XGBoost uses the learning rate when updating the model: leaf contribution = ηw_j^* .

Interpretation: leaf weight = negative sum of gradients divided by sum of Hessians + regularizer. If gradients sum to large magnitude (consistent error sign), the leaf weight is large.

6) Minimum Value Of Objective For that Tree (Useful For Split Gain)

Plugging w_j^* back into the per-leaf term yields the minimal contribution for that leaf:

$$\min_{w_j} (G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2) = -\frac{1}{2} \frac{G_j^2}{H_j + \lambda}$$

Therefore the total minimal objective for the tree (ignoring constant terms) is:

$$\widetilde{\text{Obj}}_{\min}^{(m)} = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T + \text{const}$$

This tells us how “good” the tree is; larger (more negative) the $\sum \frac{G_j^2}{H_j + \lambda}$ the better (the objective decreases more).

7) Split Gain — How XGBoost Chooses the Best Split

When considering a potential split that splits a parent node P into left L and right R , we can compute the **gain** from making the split (improvement in objective).

Parent score (if not split) would be:

$$\text{Score}(P) = \frac{1}{2} \frac{G_P^2}{H_P + \lambda}$$

Left and right scores:

$$\text{Score}(L) = \frac{1}{2} \frac{G_L^2}{H_L + \lambda}, \quad \text{Score}(R) = \frac{1}{2} \frac{G_R^2}{H_R + \lambda}$$

Gain = (score after split) – (score before) – γ (penalty per extra leaf):

$$\text{Gain} = \frac{1}{2} \left(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G_P^2}{H_P + \lambda} \right) - \gamma$$

- If $\text{Gain} > 0$ (and $>$ some threshold), the split is useful. XGBoost evaluates this for many candidate splits and picks the split with maximum Gain.
- `min_child_weight` is implemented by requiring H_L and H_R to exceed a threshold to prevent splits with too little data support.

8) Learning Rate and Update Rule

After we find the best tree structure and compute w_j^* for each leaf, the model update is:

$$\hat{y}_i^{(m)} = \hat{y}_i^{(m-1)} + \eta \cdot w_{q(x_i)}^*$$

where $0 < \eta \leq 1$. Smaller $\eta \rightarrow$ smaller steps, often better generalization but requires more trees.

9) Loss Specific Gradients & Hessians — Two Common Cases

(A) Regression (squared error)

Loss: $l(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$

- $g_i = \partial l / \partial \hat{y}_i = \hat{y}_i - y_i$
- $h_i = 1$

So for a leaf with current predictions $\hat{y}_i^{(m-1)}$:

$$G_j = \sum_{i \in I_j} (\hat{y}_i^{(m-1)} - y_i), \quad H_j = |I_j|$$

Optimal $w_j^* = -G_j / (H_j + \lambda)$. If $\hat{y}_i^{(m-1)} = 0$ initial, $w_j^* = \frac{\sum y_i}{|I_j| + \lambda}$ (i.e., essentially the mean target with shrinkage).

(B) Binary classification (logistic loss)

Use raw score F and logistic link $\sigma(F) = 1 / (1 + e^{-F})$. Loss (per sample) = negative log-likelihood:

$$l(y, F) = -[y \log \sigma(F) + (1 - y) \log(1 - \sigma(F))]$$

At current raw score $F^{(m-1)}$:

$$p_i = \sigma(F^{(m-1)}(x_i)), \quad g_i = p_i - y_i, \quad h_i = p_i(1 - p_i)$$

Then same formulas apply: compute $G_j, H_j, w_j^* = -G_j / (H_j + \lambda)$, and update raw scores by ηw_j^* . The conversion to probability is $p = \sigma(F)$.

10) Example (Tiny Numerical Walk-Through — Regression)

3 samples: $y = [3, 5, 7]$. start $\hat{y}^{(0)} = 0$ (so $g_i = -y_i$, $h_i = 1$).

- Root: $G_P = -15$, $H_P = 3$.
- Consider split: $L = \{\text{sample1 } (y=3)\}$, $R = \{\text{samples 2,3 } (y=5,7)\}$.
 - $G_L = -3$, $H_L = 1 \Rightarrow w_L^* = -(-3)/(1 + \lambda) = 3$.
 - $G_R = -12$, $H_R = 2 \Rightarrow w_R^* = 6$.
 - Parent score: $G_P^2/(H_P + \lambda) = 225/3 = 75$.
 - Left score: $9/1 = 9$. Right score: $144/2 = 72$.
 - Gain = $\frac{1}{2}(9 + 72 - 75) - \gamma = \frac{1}{2}(6) - \gamma = 3 - \gamma$. If $\gamma = 0 \rightarrow \text{Gain}=3$ (good split).
- With $\eta = 0.1$, after one tree update:
 - Predictions: sample1 $\rightarrow 0 + 0.1 \times 3 = 0.3$; sample2,3 $\rightarrow 0 + 0.1 \times 6 = 0.6$.
- Next round recompute gradients/hessians and repeat.

This numeric example shows exactly how G, H drive split decisions and leaf weights.

11) Similarity Score And Final Calculation Recap

- Per-leaf similarity score (score used in split evaluation) is:

$$\text{Score}(I) = \frac{1}{2} \cdot \frac{G_I^2}{H_I + \lambda}$$

- Split gain from parent $P \rightarrow$ children L, R :

$$\text{Gain} = \text{Score}(L) + \text{Score}(R) - \text{Score}(P) - \gamma$$

- Leaf weight:

$$w^* = -\frac{G}{H + \lambda}$$

- Objective change (minimized value) contributed by tree:

$$-\frac{1}{2} \sum_j \frac{G_j^2}{H_j + \lambda} + \gamma T$$

Remember these four concise formulas — they are the heart of XGBoost's maths.

12) Softening with L1 α and Non-Quadratic Penalties

- L1 (α) complicates closed form: leaf weight is then obtained via a **soft threshold** (coordinate-wise), not simply $-G/(H + \lambda)$. XGBoost implements this (see paper) but for intuition remember α encourages sparsity of leaf weights.
-

13) Classification: Multiclass & SoftProb

- For multiclass, XGBoost either builds K trees per boosting round (one per class) or uses vectorized derivatives. Gradients and Hessians come from softmax + cross-entropy; the same G,H aggregation and leaf formulas apply vectorially.
-

14) Algorithm Summary (Practical Pseudocode)

1. Initialize $\hat{y}_i^{(0)}$ (often 0 or log odds).
 2. For round $m = 1$ to M :
 - a. Compute g_i, h_i at $\hat{y}_i^{(m-1)}$.
 - b. Build a tree greedily: for each candidate split compute G_L, H_L, G_R, H_R and Gain.
 - c. Choose best splits until stopping (`max_depth`, `min_child_weight`, no positive gain $> \gamma$).
 - d. For each leaf compute $w_j^* = -G_j/(H_j + \lambda)$.
 - e. Update predictions: $\hat{y}_i^{(m)} = \hat{y}_i^{(m-1)} + \eta w_{q(x_i)}^*$.
 3. Final prediction: regression $\rightarrow \hat{y}$; binary $\rightarrow$ probability $\sigma(\hat{y})$; multiclass $\rightarrow$ softmax of class scores.
-

15) Key Hyperparameters and Their Roles (Explainable)

- `eta` (learning_rate): scales leaf contributions (smaller $\rightarrow$ more trees, better generalization).
 - `n_estimators`: number of boosting rounds.
 - `max_depth`: tree complexity (bigger $\rightarrow$ lowers bias, may increase variance).
 - `min_child_weight`: minimum sum of h allowed in a child (protects small splits).
 - `subsample`, `colsample_bytree`: row/column sampling (reduce correlation / variance).
 - `lambda` (L2), `alpha` (L1), `gamma` (min split gain): regularization to control overfitting.
-

16) Why the Taylor Expansion & 2nd Order Info Helps

- First-order methods (only gradients) give direction; second-order (Hessian) allows **better scaling** (Newton-like step), more accurate leaf weights, and improved split evaluation.
 - The second-order term h_i naturally downweights uncertain examples (small Hessian) and upweights confident gradients.
-

17) Practical Explanation You Can Use To Teach Someone

- **Step 1:** compute how wrong the model currently is for each sample (gradients) and how confident that error is (Hessians).
 - **Step 2:** find a tree structure whose leaves group similar errors together. For each leaf, pick a constant correction equal to (roughly) the average error scaled by confidence and regularization: $\mathbf{w} = -\mathbf{G} / (\mathbf{H} + \lambda)$.
 - **Step 3:** estimate how much the objective improves if we split a node (the gain formula), greedily pick the split that gives the biggest gain.
 - **Repeat:** add many small trees; shrink each tree's effect with η . The final model sums these small corrections.
-

18) Final Memory Cheatsheet (One Glance)

- Gradients: $g_i = \partial l / \partial \hat{y}_i$.
 - Hessians: $h_i = \partial^2 l / \partial \hat{y}_i^2$.
 - Leaf sums: $G = \sum g_i$, $H = \sum h_i$.
 - Leaf weight: $w^* = -\frac{G}{H + \lambda}$.
 - Leaf score: $\text{Score} = \frac{1}{2} \frac{G^2}{H + \lambda}$.
 - Split gain: $\frac{1}{2} \left(\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G_P^2}{H_P + \lambda} \right) - \gamma$.
 - Update: $\hat{y} \leftarrow \hat{y} + \eta w$.
-